

Turbocharging Native Code Performance

Compiler Optimizations, Profile-guided Optimizations, and Beyond

ERIC BRUMER





WAYS TO MAKE YOUR CODE RUN FASTER

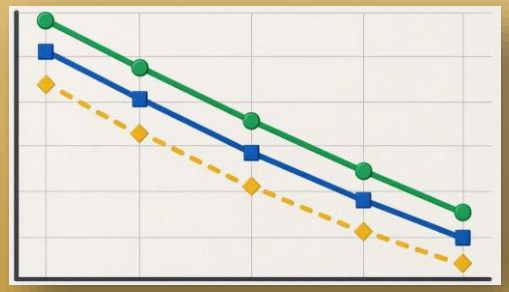
Change algorithms & data structures	Targeted modifications to your source code	Run on faster hardware	Leverage compiler



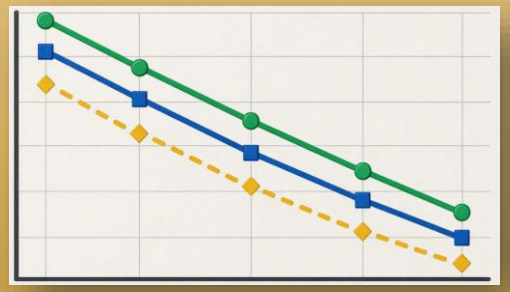
This talk, focused on CPUs

WHAT'S 'FASTER' EVEN MEAN?

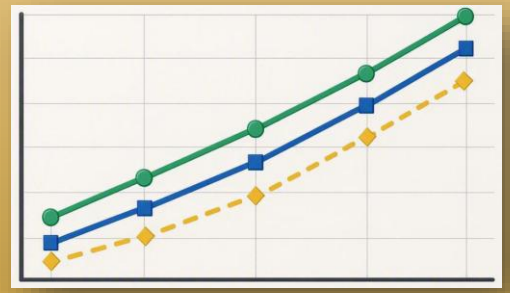
Program Execution Time



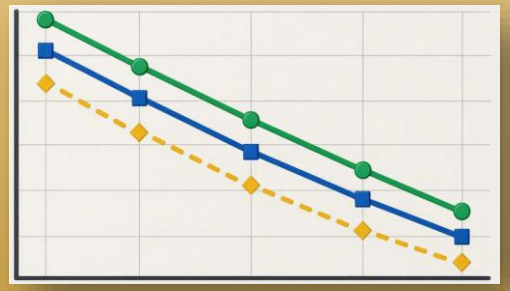
Latency



Throughput



CPU Utilization



AGENDA

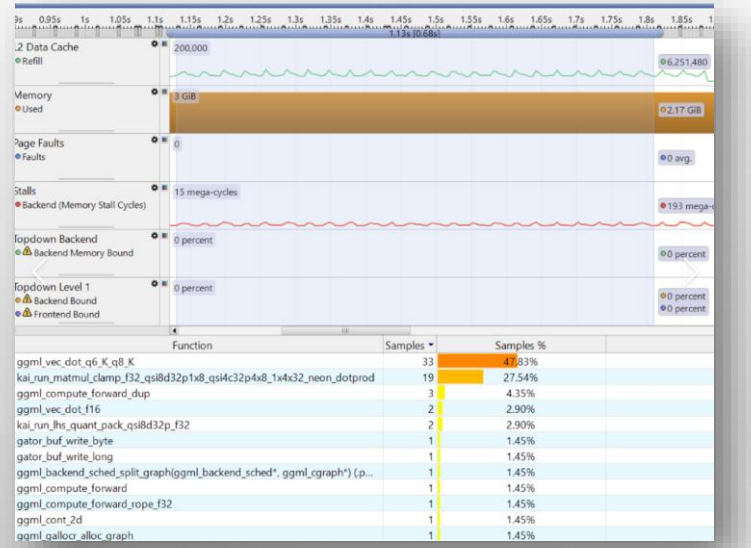
1. The tools: profilers
2. The context: CPUs, and Eric's performance corollary
3. The optimizations
    - Compiler optimizations
    - Profile-guided optimizations

AGENDA

1. The tools: profilers
2. The context: CPUs, and Eric's performance corollary
3. The optimizations
    - Compiler optimizations
    - Profile-guided optimizations

PROFILERS

Use ones that read hardware performance counters

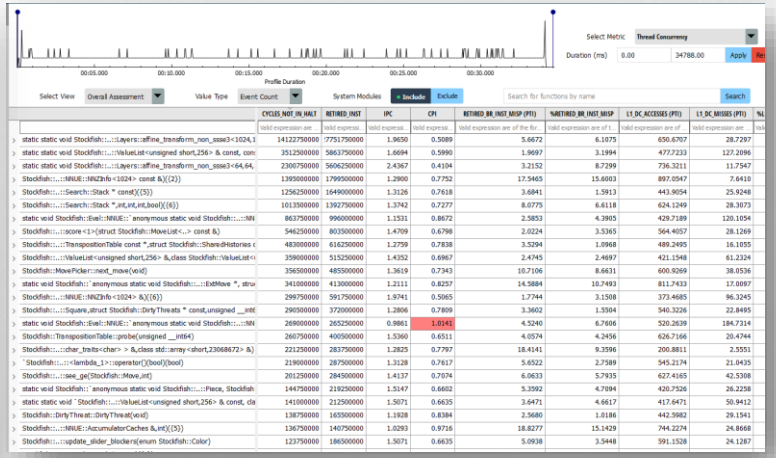


ARM Streamline (Linux/Elf)

https://support.arm.com/tools-and-software/streamline-performance-analyzer

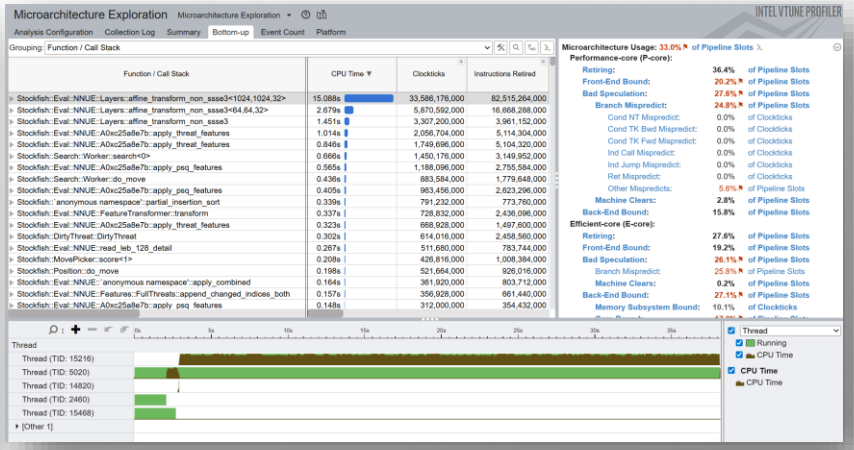
WindowsPerf (Windows/PE binaries)

https://learn.arm.com/install-guides/wperf/



AMD uProf

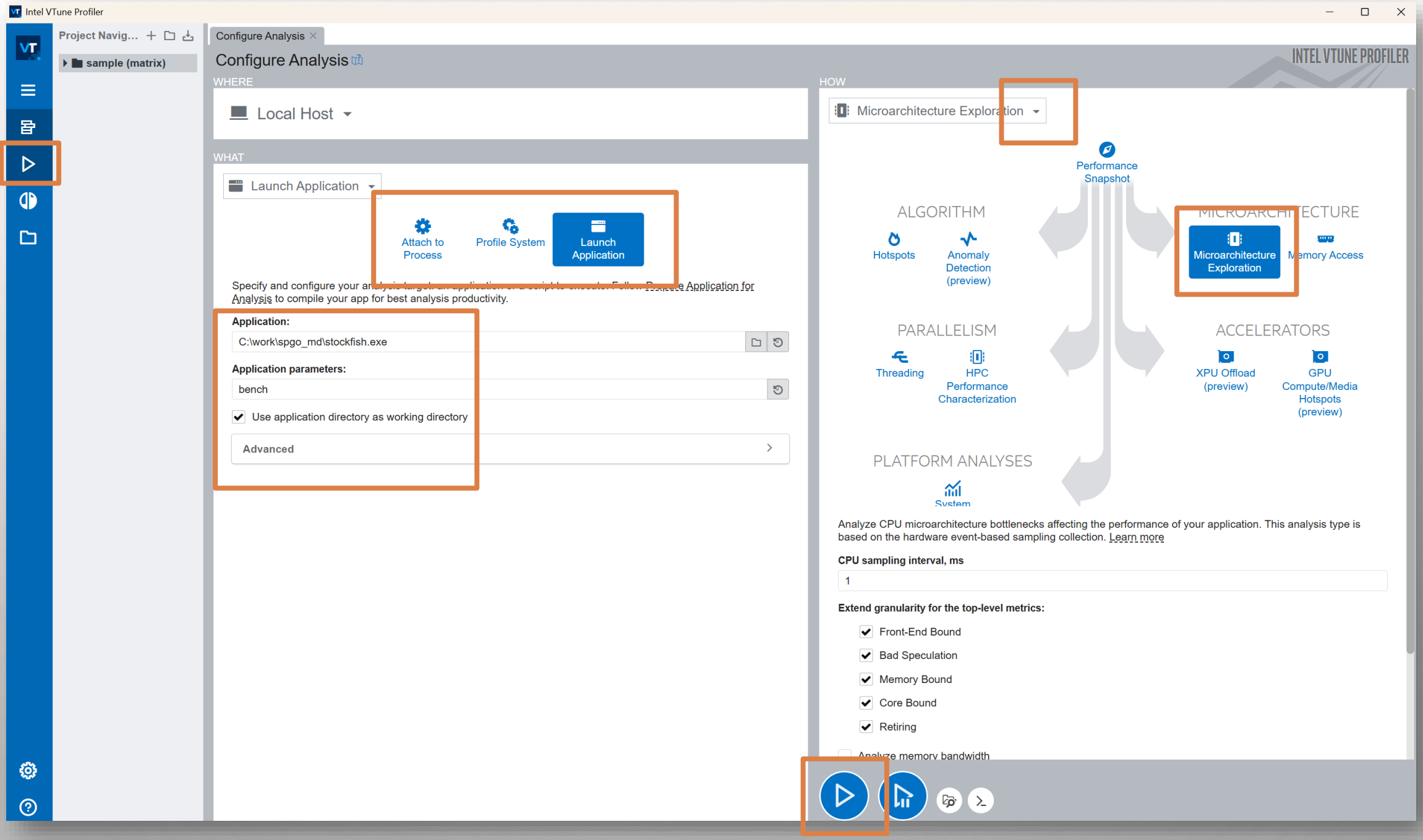
https://www.amd.com/en/developer/uprof.html

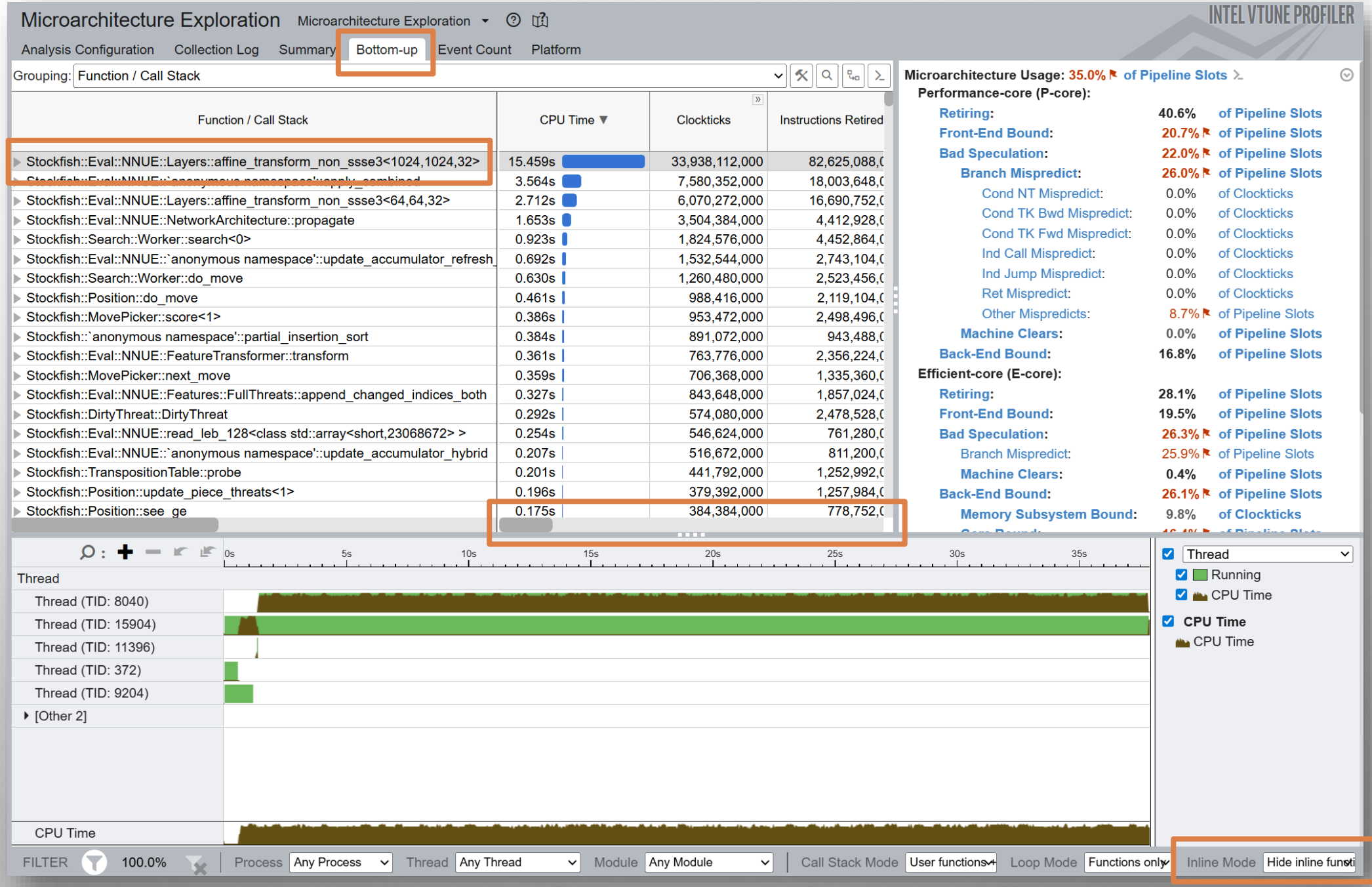


Intel VTune

https://www.intel.com/content/www/us/en/developer/tools/oneapi/vtune-profiler-download.html

Intel VTune


Intel VTune Profiler
Project Navig...
sample (matrix)
Configure Analysis
Configure Analysis
INTEL VTUNE PROFILER
WHERE
Local Host
WHAT
Launch Application
Attach to Process
Profile System
Launch Application
Specify and configure your analysis target: an application or a script to execute. Follow Prepare Application for Analysis to compile your app for best analysis productivity.
Application:
C:\work\spgo_md\stockfish.exe
Application parameters:
bench
Use application directory as working directory
Advanced
HOW
Microarchitecture Exploration
Performance Snapshot
ALGORITHM
Hotspots
Anomaly Detection (preview)
MICROARCHITECTURE
Microarchitecture Exploration
Memory Access
PARALLELISM
Threading
HPC Performance Characterization
ACCELERATORS
XPU Offload (preview)
GPU Compute/Media Hotspots (preview)
PLATFORM ANALYSES
System
Analyze CPU microarchitecture bottlenecks affecting the performance of your application. This analysis type is based on the hardware event-based sampling collection. Learn more
CPU sampling interval, ms
1
Extend granularity for the top-level metrics:
Front-End Bound
Bad Speculation
Memory Bound
Core Bound
Retiring
Analyze memory bandwidth


Microarchitecture Exploration
Microarchitecture Exploration
INTEL VTUNE PROFILER
Analysis Configuration
Collection Log
Summary
Bottom-up
Event Count
Platform
Grouping: Function / Call Stack
Function / Call Stack
CPU Time
Clockticks
Instructions Retired
Stockfish::Eval::NNUE::Layers::affine_transform_non_ssse3<1024,1024,32> 15.459s 33,938,112,000 82,625,088,(
3.564s 7,580,352,000 18,003,648,(
Stockfish::Eval::NNUE::Layers::affine_transform_non_ssse3<64,64,32> 2.712s 6,070,272,000 16,690,752,(
Stockfish::Eval::NNUE::NetworkArchitecture::propagate 1.653s 3,504,384,000 4,412,928,(
Stockfish::Search::Worker::search<0> 0.923s 1,824,576,000 4,452,864,(
Stockfish::Eval::NNUE::`anonymous namespace'::update_accumulator_refresh 0.692s 1,532,544,000 2,743,104,(
Stockfish::Search::Worker::do_move 0.630s 1,260,480,000 2,523,456,(
Stockfish::Position::do_move 0.461s 988,416,000 2,119,104,(
Stockfish::MovePicker::score<1> 0.386s 953,472,000 2,498,496,(
Stockfish::`anonymous namespace'::partial_insertion_sort 0.384s 891,072,000 943,488,(
Stockfish::Eval::NNUE::FeatureTransformer::transform 0.361s 763,776,000 2,356,224,(
Stockfish::MovePicker::next_move 0.359s 706,368,000 1,335,360,(
Stockfish::Eval::NNUE::Features::FullThreats::append_changed_indices_both 0.327s 843,648,000 1,857,024,(
Stockfish::DirtyThreat::DirtyThreat 0.292s 574,080,000 2,478,528,(
Stockfish::Eval::NNUE::read_leb_128<class std::array<short,23068672> > 0.254s 546,624,000 761,280,(
Stockfish::Eval::NNUE::`anonymous namespace'::update_accumulator_hybrid 0.207s 516,672,000 811,200,(
Stockfish::TranspositionTable::probe 0.201s 441,792,000 1,252,992,(
Stockfish::Position::update_piece_threats<1> 0.196s 379,392,000 1,257,984,(
Stockfish::Position::see_ge 0.175s 384,384,000 778,752,(
Microarchitecture Usage: 35.0% of Pipeline Slots
Performance-core (P-core):
Retiring: 40.6% of Pipeline Slots
Front-End Bound: 20.7% of Pipeline Slots
Bad Speculation: 22.0% of Pipeline Slots
Branch Mispredict: 26.0% of Pipeline Slots
Cond NT Mispredict: 0.0% of Clockticks
Cond TK Bwd Mispredict: 0.0% of Clockticks
Cond TK Fwd Mispredict: 0.0% of Clockticks
Ind Call Mispredict: 0.0% of Clockticks
Ind Jump Mispredict: 0.0% of Clockticks
Ret Mispredict: 0.0% of Clockticks
Other Mispredicts: 8.7% of Pipeline Slots
Machine Clears: 0.0% of Pipeline Slots
Back-End Bound: 16.8% of Pipeline Slots
Efficient-core (E-core):
Retiring: 28.1% of Pipeline Slots
Front-End Bound: 19.5% of Pipeline Slots
Bad Speculation: 26.3% of Pipeline Slots
Branch Mispredict: 25.9% of Pipeline Slots
Machine Clears: 0.4% of Pipeline Slots
Back-End Bound: 26.1% of Pipeline Slots
Memory Subsystem Bound: 9.8% of Clockticks
0s 5s 10s 15s 20s 25s 30s 35s
Thread
Thread (TID: 8040)
Thread (TID: 15904)
Thread (TID: 11396)
Thread (TID: 372)
Thread (TID: 9204)
[Other 2]
CPU Time
Thread
Running
CPU Time
CPU Time
CPU Time
FILTER 100.0%
Process Any Process
Thread Any Thread
Module Any Module
Call Stack Mode User functions
Loop Mode Functions only
Inline Mode Hide inline funct

AMD UPROF

AGENDA

1. The tools: profilers
2. The context: CPUs, and Eric's performance corollary
3. The optimizations
    - Compiler optimizations
    - Profile-guided optimizations

2.5 Processor Block Diagram

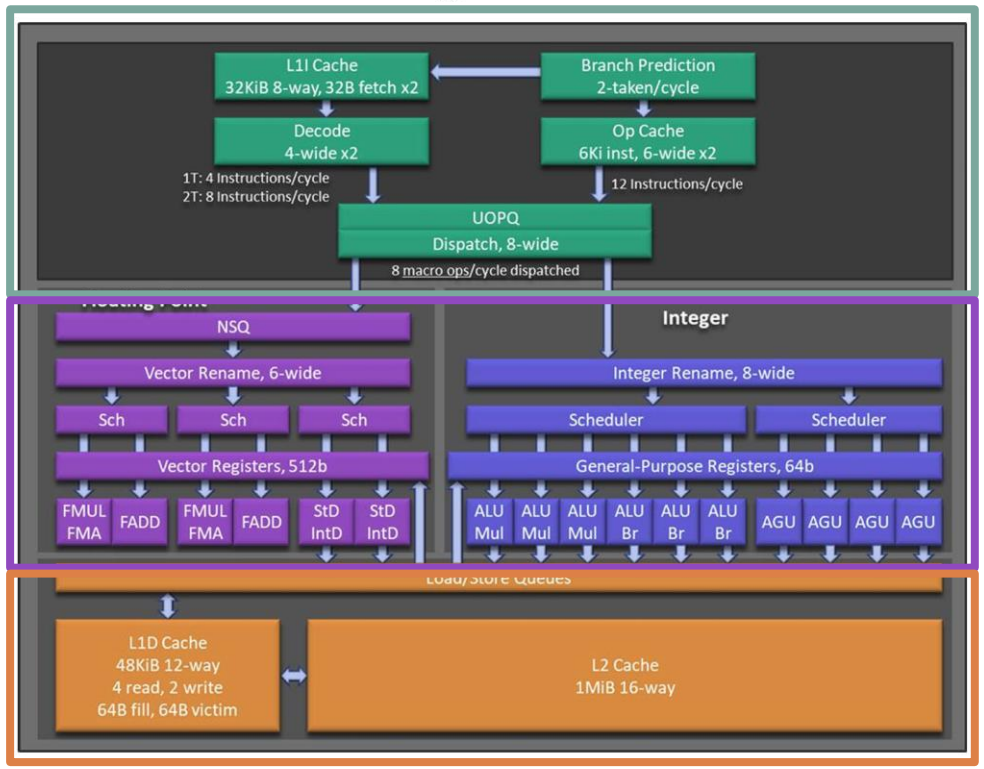


Figure 1. AMD Zen5 CPU Core Block Diagram

Zen5 Software Optimization Guide

https://docs.amd.com/v/u/en-US/58455_1.00

Hardware performance	Compiler optimizations
CPU Front-End • Instruction cache • Branch prediction	• Code layout • Devirtualization
CPU Back-End • Integer units • FP units • Vector units	• Emit efficient instruction sequences • SIMD generation
Memory system • Data cache • L2 shared cache	• Generate fewer loads on critical paths • Data layout

A word on stalls

- Hardware performance counters tell you stalls.
- Performance tuning is whack-a-mole.

Eric's performance corollary: all sufficiently optimized code is memory bound.

AGENDA

1. The tools: profilers
2. The context: CPUs, and Eric's performance corollary
3. The optimizations
    - Compiler optimizations
    - Profile-guided optimizations

COMPILER OPTIMIZATIONS

- About: how the compiler improves the performance of your code.
- Not about: “how to write your code to be faster”

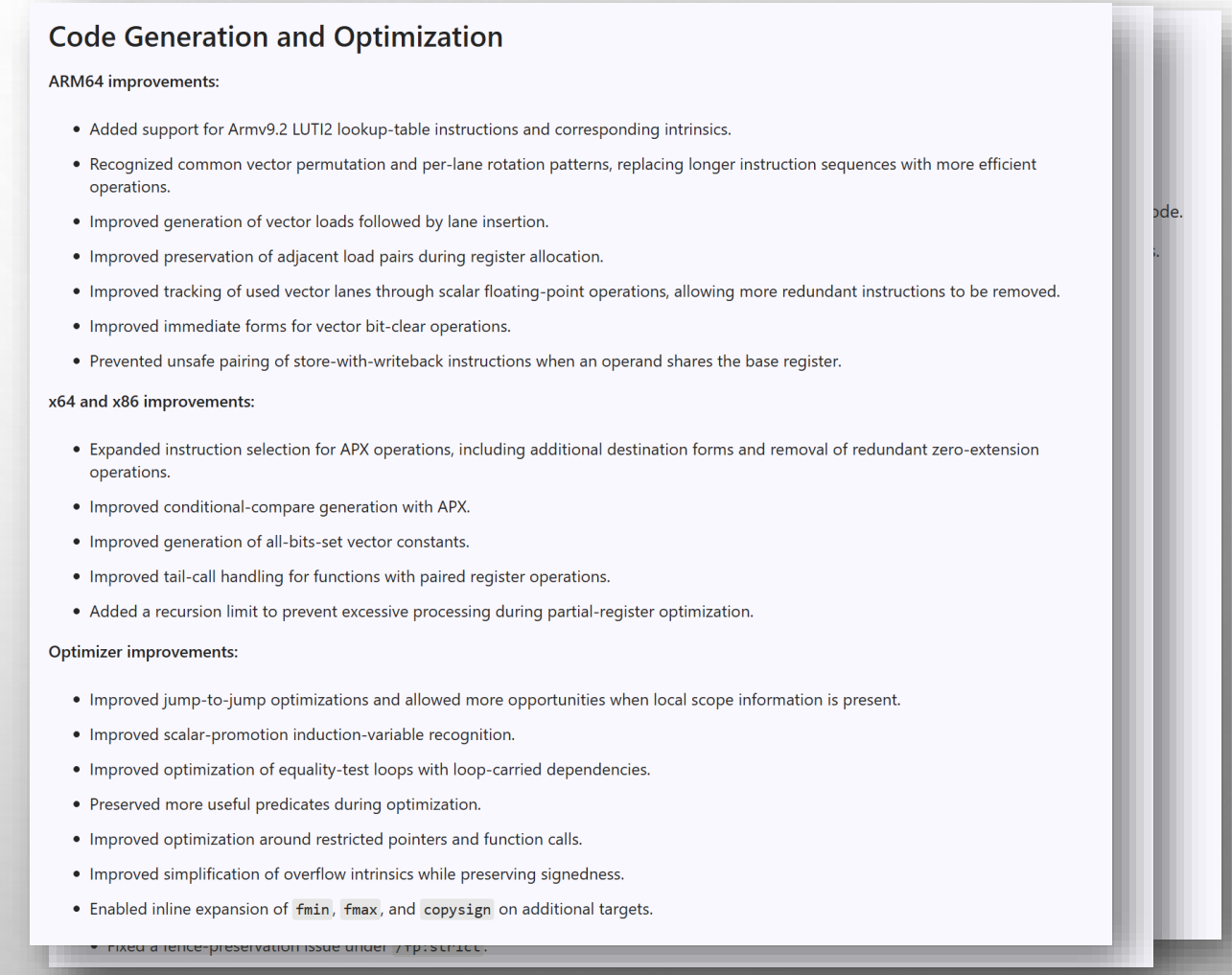
CHECK THE DOCS!

New in 14.51

- New SSA loop optimizer
- New SLP vectorizer
- SROA assignments through indirections
- SROA support for larger structures
- SROA support for unions
- Relaxed address-taken SROA restrictions
- SROA with casted structure fields
- SROA with non-zero source offsets
- Structure repacking through indirections
- Hoisted vectorizer pointer-overlap checks
- Logical OR converted to bitwise OR
- Shift and comparison folding
- Improved ARM64 NEON code generation
- Unconditional store execution
- Improved AVX optimization
- Single and limited call-site inlining
- Branch elimination through nested conditions
- Iterator-loop unswitching
- Improved memset and memcpy
- ARM64 bitwise operations with shifted registers
- Ternary operator optimization
- Improved copy propagation
- Expanded complete loop unrolling

14.52 July Update

14.52 June Update



Code Generation and Optimization

ARM64 improvements:

- Added support for Armv9.2 LUTI2 lookup-table instructions and corresponding intrinsics.
- Recognized common vector permutation and per-lane rotation patterns, replacing longer instruction sequences with more efficient operations.
- Improved generation of vector loads followed by lane insertion.
- Improved preservation of adjacent load pairs during register allocation.
- Improved tracking of used vector lanes through scalar floating-point operations, allowing more redundant instructions to be removed.
- Improved immediate forms for vector bit-clear operations.
- Prevented unsafe pairing of store-with-writeback instructions when an operand shares the base register.

x64 and x86 improvements:

- Expanded instruction selection for APX operations, including additional destination forms and removal of redundant zero-extension operations.
- Improved conditional-compare generation with APX.
- Improved generation of all-bits-set vector constants.
- Improved tail-call handling for functions with paired register operations.
- Added a recursion limit to prevent excessive processing during partial-register optimization.

Optimizer improvements:

- Improved jump-to-jump optimizations and allowed more opportunities when local scope information is present.
- Improved scalar-promotion induction-variable recognition.
- Improved optimization of equality-test loops with loop-carried dependencies.
- Preserved more useful predicates during optimization.
- Improved optimization around restricted pointers and function calls.
- Improved simplification of overflow intrinsics while preserving signedness.
- Enabled inline expansion of `fmin`, `fmax`, and `copysign` on additional targets.
- Fixed a fence-preservation issue under `/fp:strict`.

https://devblogs.microsoft.com/cppblog/

CHECK THE DOCS!

GCC Timeline

The table is sorted by date. Please refer to our development plan f

Release	Release date
GCC 16.2	August 7, 2026
GCC 14.4	June 26, 2026
GCC 15.3	June 12, 2026
GCC 16.1	April 30, 2026
GCC 15.2	August 8, 2025
GCC 12.5	July 11, 2025
GCC 13.4	June 5, 2025
GCC 14.3	May 23, 2025
GCC 15.1	April 25, 2025
GCC 14.2	August 1, 2024
GCC 11.5	July 19, 2024
GCC 12.4	June 20, 2024
GCC 13.3	May 21, 2024
GCC 14.1	May 7, 2024
GCC 13.2	July 27, 2023
GCC 10.5	July 7, 2023

https://gcc.gnu.org/releases.html

Download

LLVM provides extensive documentation in HTML form, which is available in the source download and online.

Date	Version	Download	Release Notes	Documentation
Current	Git	via Git	release notes	docs
25 Jul 2026	23.1.0	download	llvm clang lld clang-extra libc++ polly flang	llvm clang lld clang-extra libc++ polly flang
16 Jun 2026	22.1.8	download	llvm clang lld clang-extra libc++ polly flang	llvm clang lld clang-extra libc++ polly flang
10 Mar 2026	22.1.1	download	llvm clang lld clang-extra libc++ polly flang	llvm clang lld clang-extra libc++ polly flang
24 Feb 2026	22.1.0	download	llvm clang lld clang-extra libc++ polly flang	llvm clang lld clang-extra libc++ polly flang
18 Nov 2025	21.1.6	download	llvm clang lld clang-extra libc++ polly flang	llvm clang lld clang-extra libc++ polly flang
26 Aug 2025	21.1.0	download	llvm clang lld clang-extra libc++ polly flang	llvm clang lld clang-extra libc++ polly flang
04 Mar 2025	20.1.0	download	llvm clang lld clang-extra libc++ polly flang	llvm clang lld clang-extra libc++ polly flang
1 Oct 2024	19.1.1	download	release notes	
17 Sep 2024	19.1.0	download	llvm clang lld clang-extra libc++ polly flang	llvm clang lld clang-extra libc++ polly flang
26 Jul 2024	19.1.0-rc1	download	release notes	
20 Jun 2024	18.1.8	download	release notes	
6 Jun 2024	18.1.7	download	release notes	

https://releases.llvm.org/

COMPILER OPTIMIZATIONS

- Want new optimizations? grab a new compiler!
- We're always making things faster for you.

PERF RESULTS: SPEC CPU® 2017

Comparing MSVC 14.50 to 14.51 (~6 months)

https://devblogs.microsoft.com/cppblog/c-performance-improvements-in-msvc-build-tools-v14-51/

Suite	MSVC Config	x64	Arm64
SPECspeed® 2017_int_base (est.)	Peak (PGO)	5.0% faster	6.5% faster
SPECspeed® 2017_int_base (est.)	VS Defaults	4.3% faster	4.4% faster

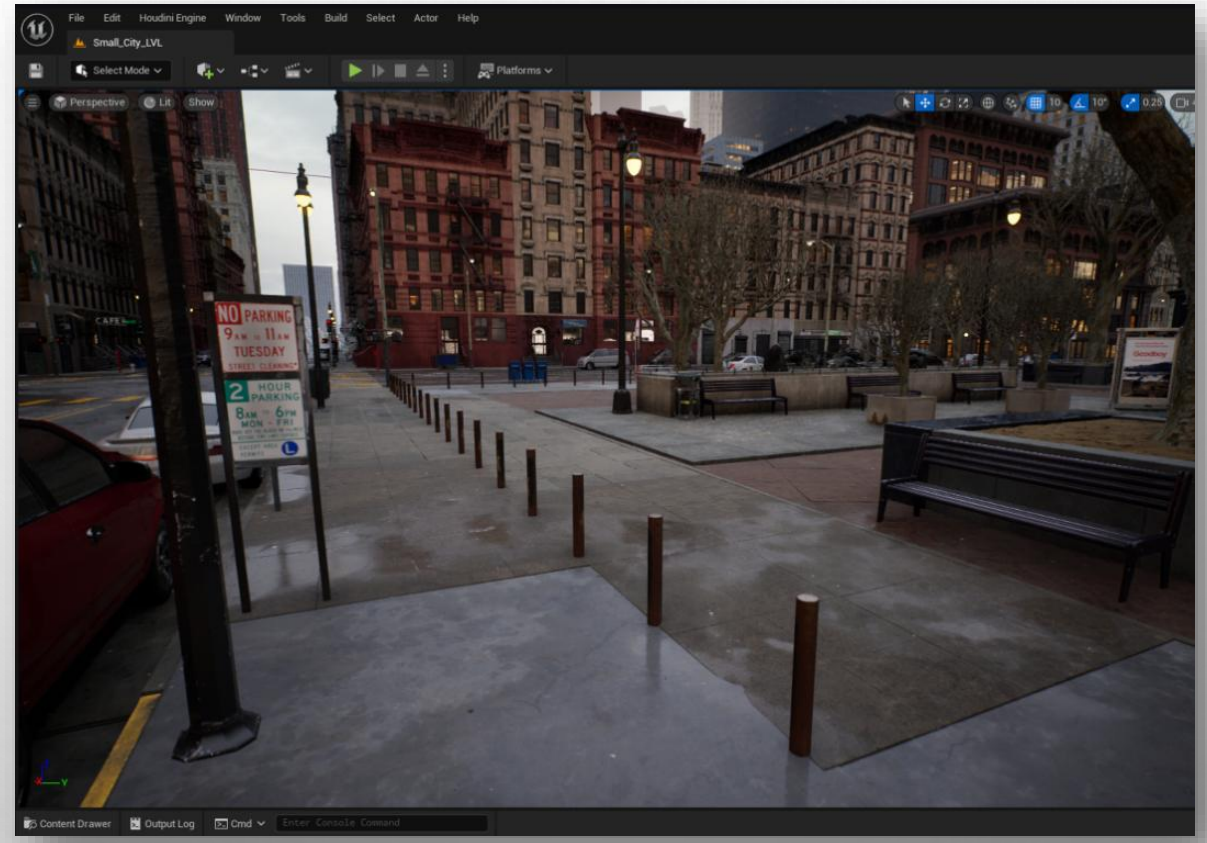
⚠ **Note:**

SPEC®, SPEC CPU®, and SPECspeed® are trademarks of the Standard Performance Evaluation Corporation (SPEC).

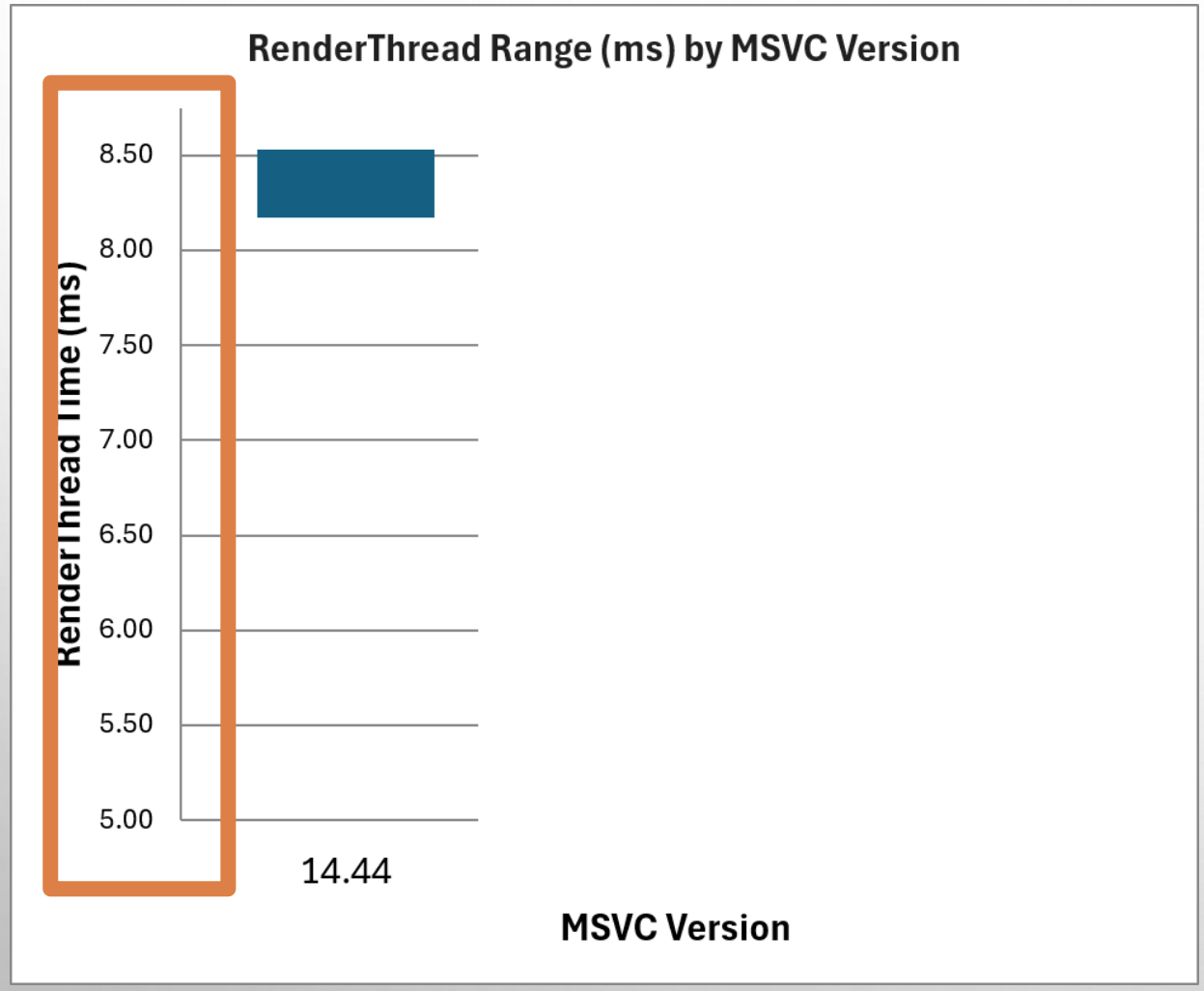
Does not include Fortran benchmarks; results are *estimated*.

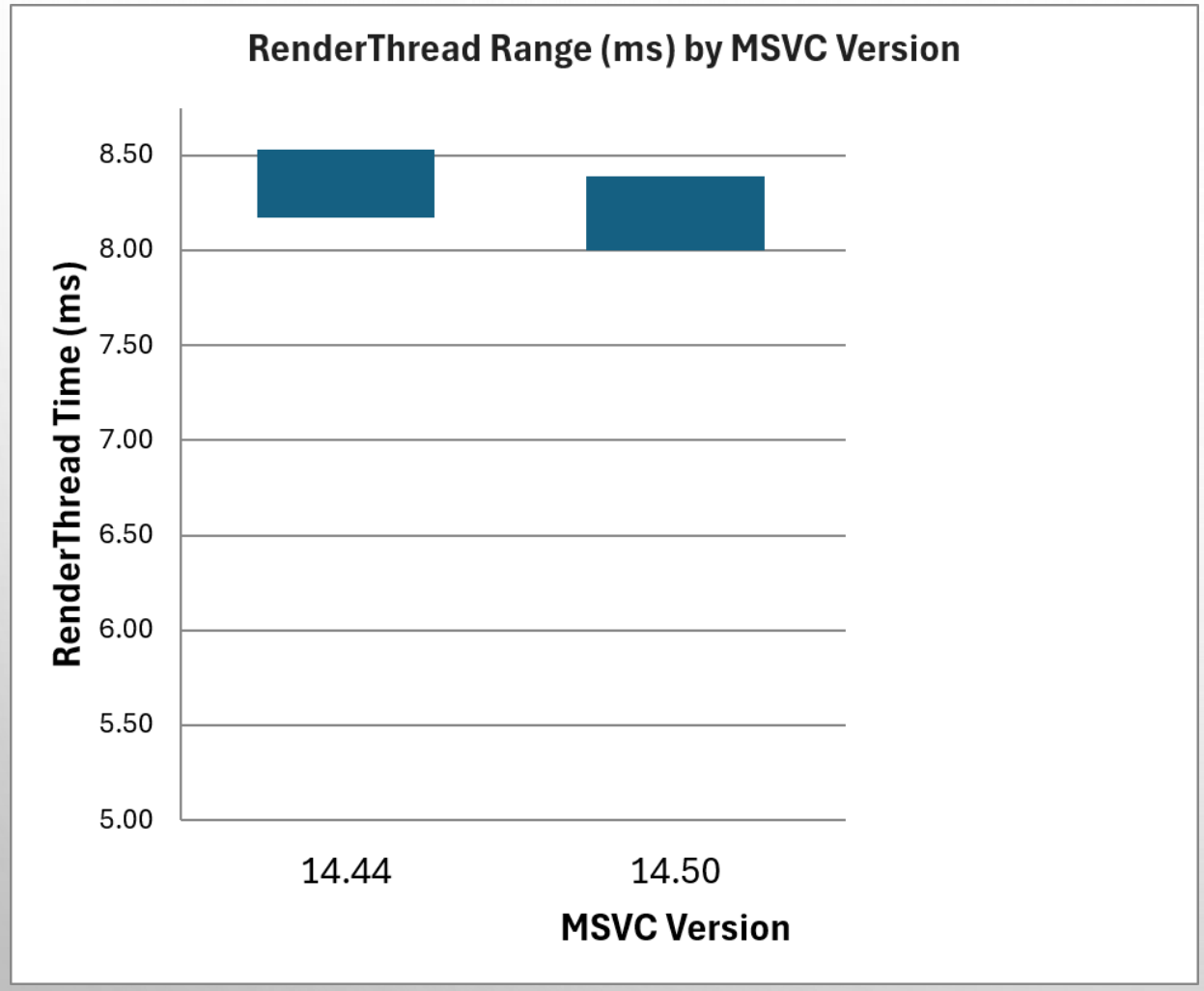
https://www.spec.org/

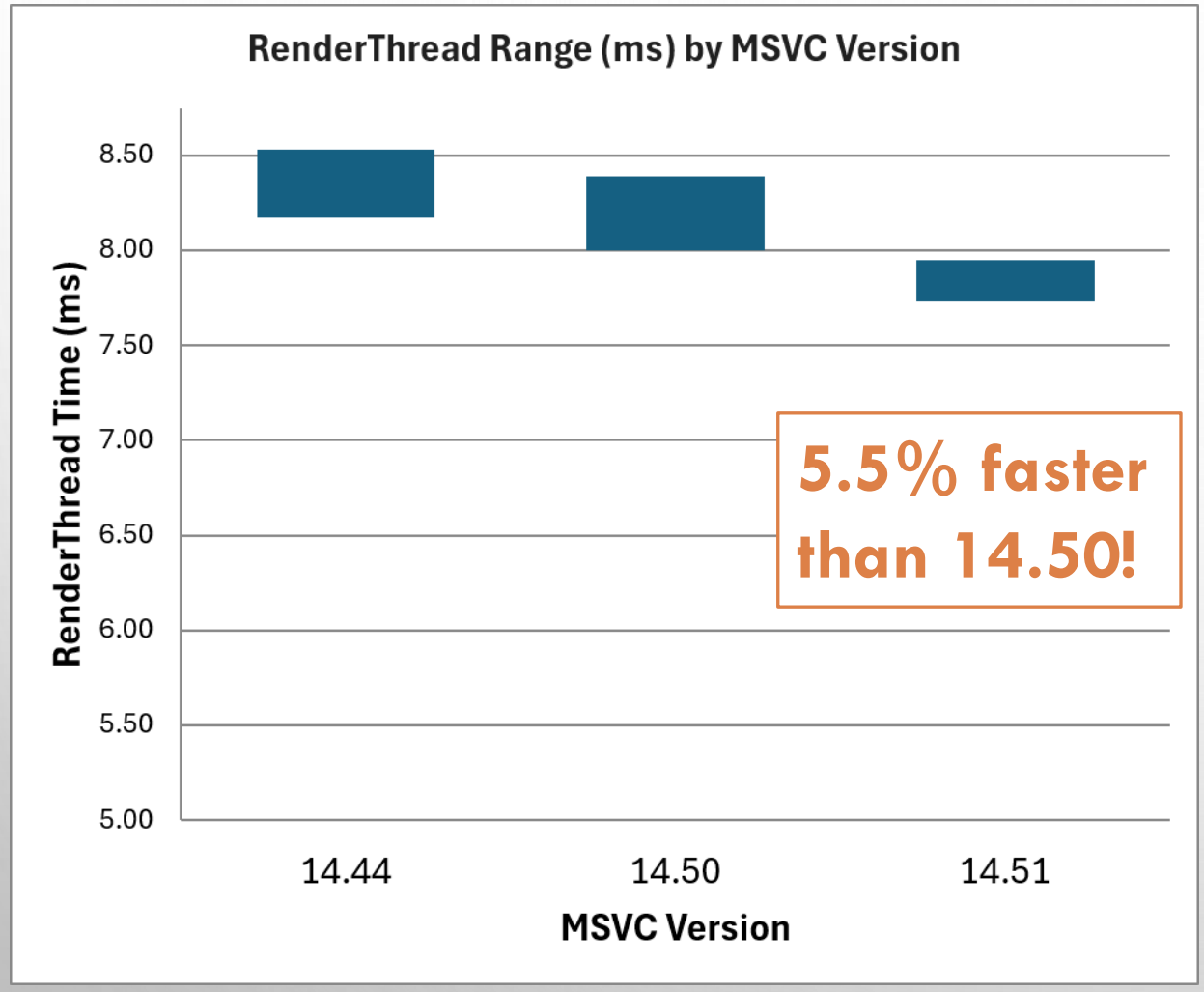
PERF RESULTS: UNREAL ENGINE CITY SAMPLE

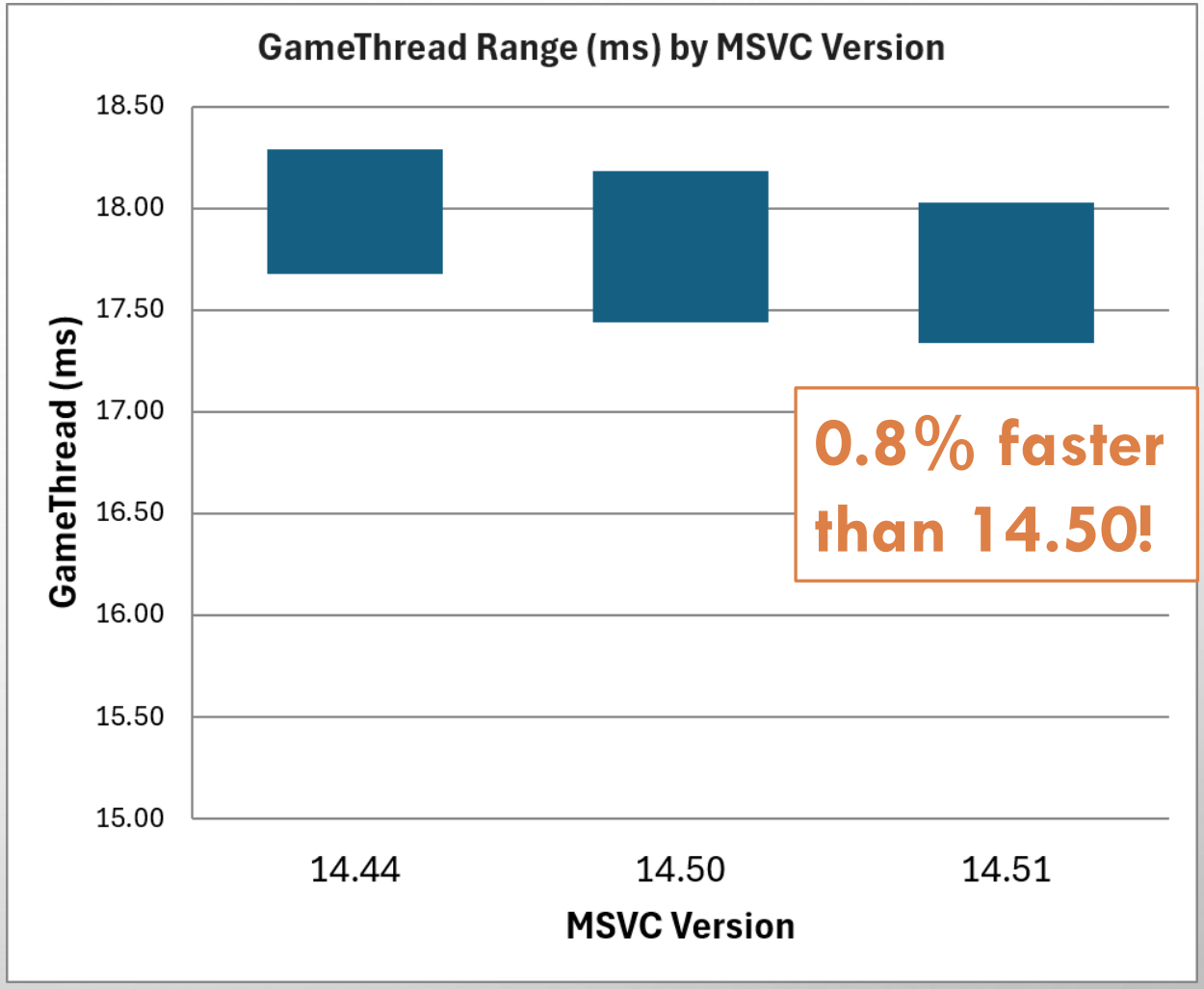


https://dev.epicgames.com/documentation/unreal-engine/city-sample-project-unreal-engine-demonstration


RenderThread Range (ms) by MSVC Version
RenderThread Time (ms)
8.50
8.00
7.50
7.00
6.50
6.00
5.50
5.00
14.44
MSVC Version


RenderThread Range (ms) by MSVC Version
RenderThread Time (ms)
8.50
8.00
7.50
7.00
6.50
6.00
5.50
5.00
14.44
14.50
MSVC Version

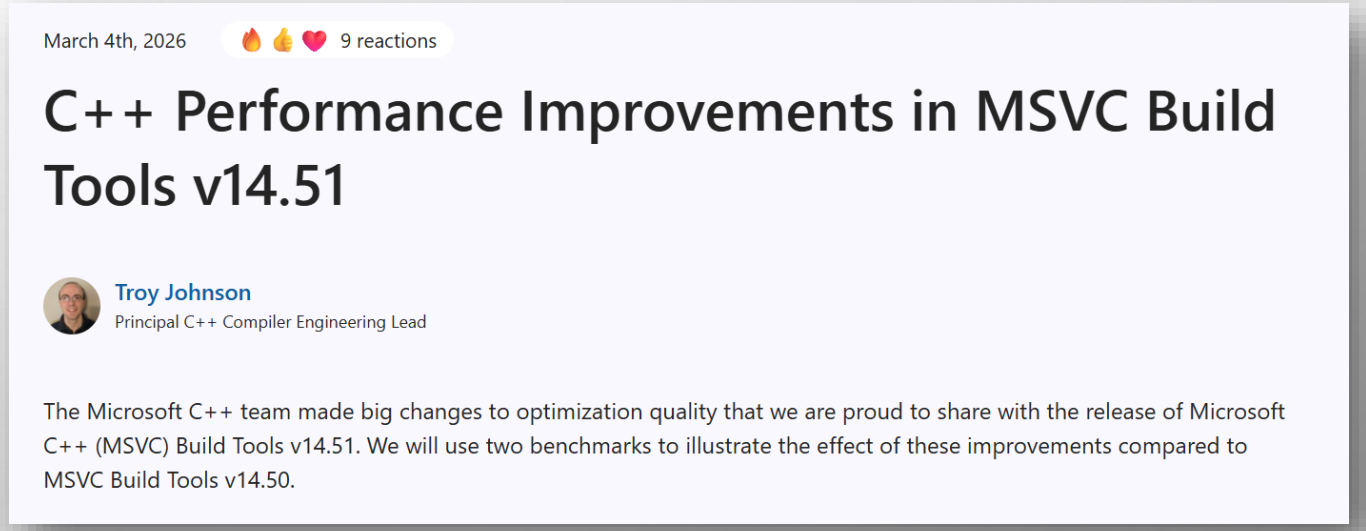

RenderThread Range (ms) by MSVC Version
RenderThread Time (ms)
8.50
8.00
7.50
7.00
6.50
6.00
5.50
5.00
14.44
14.50
14.51
MSVC Version
5.5% faster than 14.50!


GameThread Range (ms) by MSVC Version
GameThread (ms)
18.50
18.00
17.50
17.00
16.50
16.00
15.50
15.00
14.44
14.50
14.51
MSVC Version
0.8% faster than 14.50!

COMPILER OPTIMIZATIONS

https://devblogs.microsoft.com/cppblog/c-performance-improvements-in-msvc-build-tools-v14-51/

- Permute/broadcast
- SLP
- SROA



PERMUTE/BROADCAST IDENTIFICATION #1

```
// Compile with /O2 for ARM64
void test(int* __restrict a, int* __restrict b)
{
    a[0] = b[0] + b[5];
    a[1] = b[1] - b[4];
    a[2] = b[2] + b[7];
    a[3] = b[3] - b[6];
    a[4] = b[4] + b[1];
    a[5] = b[5] - b[0];
    a[6] = b[6] + b[3];
    a[7] = b[7] - b[2];
}
```

Wins across CPU front-end

- Fewer instructions

Wins across CPU back-end

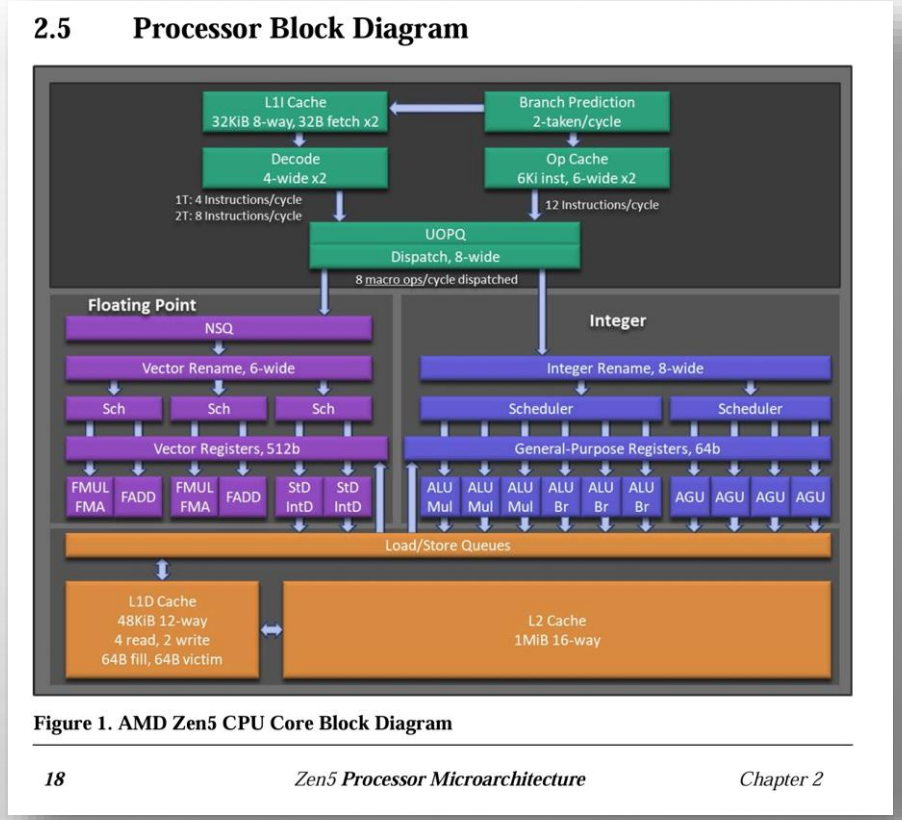
- SIMD generation
- Fewer load & store instrs.

MSVC 14.50

```
|void test(int *,int *)|
    ldp         w6,w7,[x1]
    ldp         w14,w15,[x1,#0x10]
    ldp         w13,w12,[x1,#8]
    ldp         w10,w11,[x1,#0x18]
    add         w9,w6,w15
    sub         w8,w7,w14
    stp         w9,w8,[x0]
    add         w9,w13,w11
    sub         w8,w12,w10
    stp         w9,w8,[x0,#8]
    add         w9,w14,w7
    sub         w8,w15,w6
    stp         w9,w8,[x0,#0x10]
    add         w9,w10,w12
    sub         w8,w11,w13
    stp         w9,w8,[x0,#0x18]
    ret
```

MSVC 14.51

```
|void test(int *,int *)|
    ldp         q20,q16,[x1]
    uzp2        v17.4s,v16.4s,v20.4s
    uzp1        v18.4s,v20.4s,v16.4s
    uzp2        v19.4s,v20.4s,v16.4s
    uzp1        v16.4s,v16.4s,v20.4s
    add         v18.4s,v18.4s,v17.4s
    sub         v16.4s,v19.4s,v16.4s
    zip1        v17.4s,v18.4s,v16.4s
    zip2        v16.4s,v18.4s,v16.4s
    stp         q17,q16,[x0]
    ret
```



2.5 Processor Block Diagram

Figure 1. AMD Zen5 CPU Core Block Diagram

PERMUTE/BROADCAST IDENTIFICATION #2

```cpp
#include <immintrin.h>

// Compile with /O2 /arch:AVX
__m256d test(double *a, double *b, double *c, double *d) {
    __m256d t1 = _mm256_set_pd(*d, *c, *b, *a);      // t1 = {*d, *c, *b, *a}
    __m256d t2 = _mm256_permute2f128_pd(t1, t1, 32); // t2 = {*b, *a, *b, *a}
    __m256d t3 = _mm256_movedup_pd(t2);              // t3 = {*a, *a, *a, *a}
    return t3;
}
```

MSVC 14.50

```
__m256d test(double *) PROC
    vmovsd      xmm0, QWORD PTR [rdx]
    vmovsd      xmm1, QWORD PTR [rcx]
    vmovsd      xmm2, QWORD PTR [r9]
    vmovsd      xmm3, QWORD PTR [r8]
    vunpcklpd   xmm1, xmm1, xmm0
    vunpcklpd   xmm3, xmm3, xmm2
    vinsertf128 ymm0, ymm1, xmm3, 1
    vperm2f128  ymm0, ymm0, ymm0, 32
    vmovddup    ymm0, ymm0
    ret     0
```

MSVC 14.51

```
__m256d test(double *) PROC
  vbroadcastsd ymm0, QWORD PTR [rcx]
  ret     0
```

This yielded almost a single FPS in the Unreal Engine.

Wins across CPU front-end

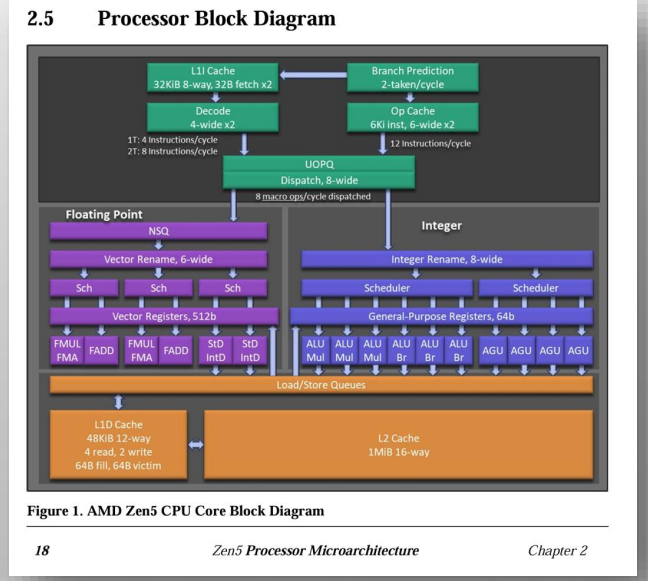
- Fewer instructions

Wins across CPU back-end

- More efficient instruction sequences
- Fewer load instrs

Wins across memory system

- Less memory usage



SROA IMPROVEMENTS

```
struct S {
    int a; int b; int c;
    int d; int e; int f;
    int g; int h; int i;
};

// Compile with /O2 for x64
int test2(bool flag) {
   S s1;
   S s2;
   s1.a = 1;
   s2.a = 1;

   S s3 = s1;
   if (flag) s3 = s2;

   return s3.a;
}
```

Wins across CPU front-end

- Fewer instructions

Wins across memory system

- Less memory usage

MSVC 14.50

```
int test2(bool) PROC
  sub     rsp, 136
  mov     DWORD PTR s1$[rsp], 1
  mov     DWORD PTR s2$[rsp], 1
  mov     eax, DWORD PTR s2$[rsp]
  movups  xmm1, XMMWORD PTR s1$[rsp]
  test    cl, cl
  jne     SHORT $LN2@test2
  movd    eax, xmm1
$LN2@test2:
  add     rsp, 136
  ret     0
```

MSVC 14.51

```
int test2(bool) PROC
  mov     eax, 1
  ret     0
```

2.5 Processor Block Diagram

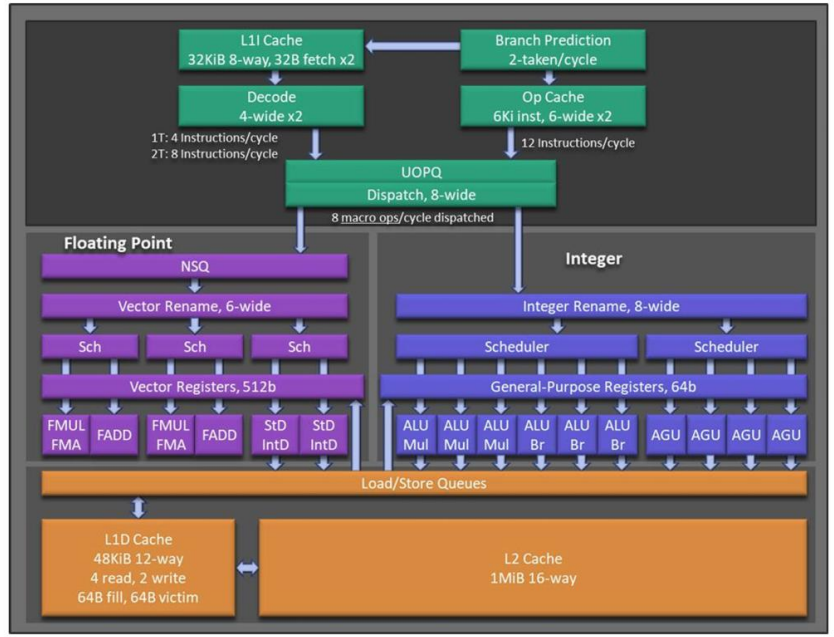


Figure 1. AMD Zen5 CPU Core Block Diagram

COMPILER OPTIMIZATIONS

Many optimizations affect multiple parts of the CPU

- Significant inliner heuristic changes
- New loop optimizer
- Unswitching now kicks in for more cases
- https://devblogs.microsoft.com/cppblog/

Want faster code?

- Upgrade compilers!

AGENDA

1. The tools: profilers
2. The context: CPUs, and Eric's performance corollary
3. The optimizations
    - Compiler optimizations
    - Profile-guided optimizations

PGO: PROFILE-GUIDED-OPTIMIZATIONS

- Insight: a subset of your code is ‘hot’, the majority is ‘cold’
- The process:
    1. Collect a profile trace
    2. Feed the trace back to the next compilation
        - Hyper-optimize the hot code
        - Cold code is laid out separately
- Technology: PGO (MSVC), FDO (GCC & Clang)

https://aka.ms/spgo/tutorial & https://aka.ms/spgo/blog

SAMPLE-BASED



SPGO: PROFILE-GUIDED-OPTIMIZATIONS

- Insight: a subset of your code is 'hot', the majority is 'cold'
- The process:
    1. Collect a profile trace via sampling
    2. Feed the trace back to the next compilation
        - Hyper-optimize the hot code
        - Cold code is laid out separately
- ~~Technology: PGO (MSVC), FDO (GCC & Clang)~~
- Technology: SPGO (MSVC), AutoFDO (GCC & Clang)

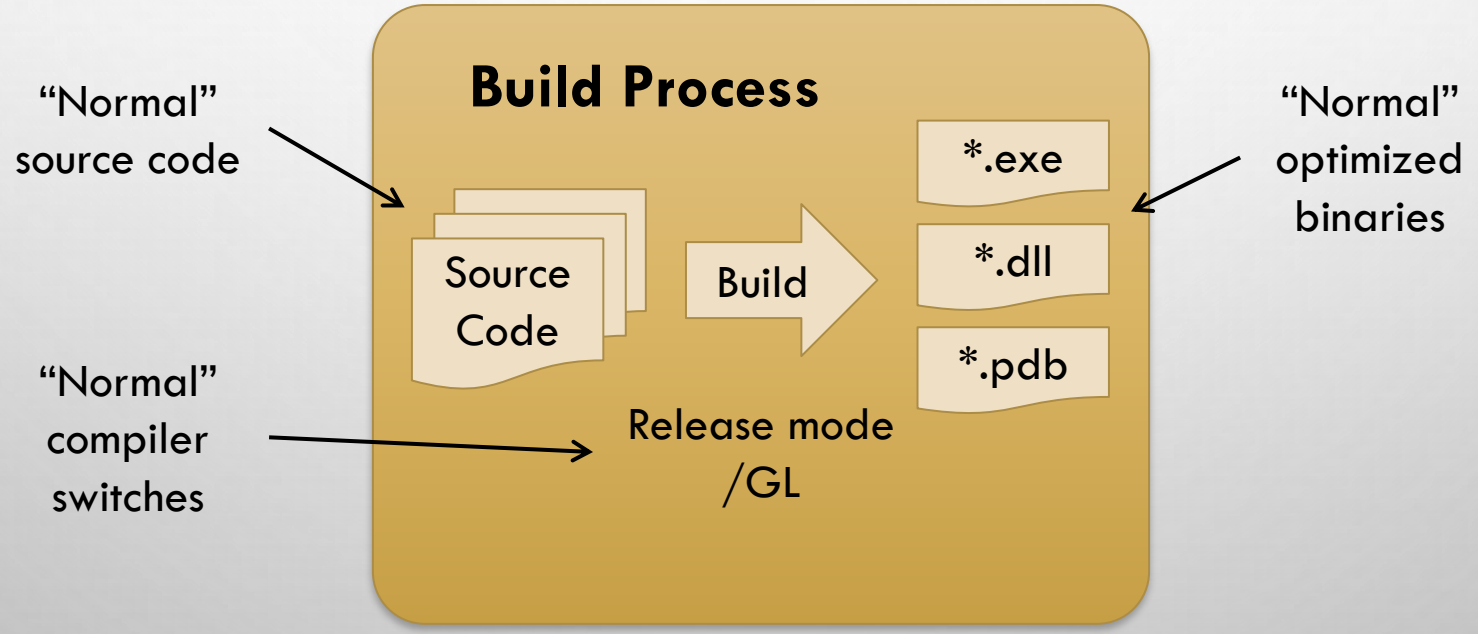
https://aka.ms/spgo/tutorial & https://aka.ms/spgo/blog

SPGO

https://aka.ms/spgo/tutorial & https://aka.ms/spgo/blog

Available on Visual Studio 2022 & later: 14.44, 14.50, 14.51, etc

5-15% performance improvements (YMMV)

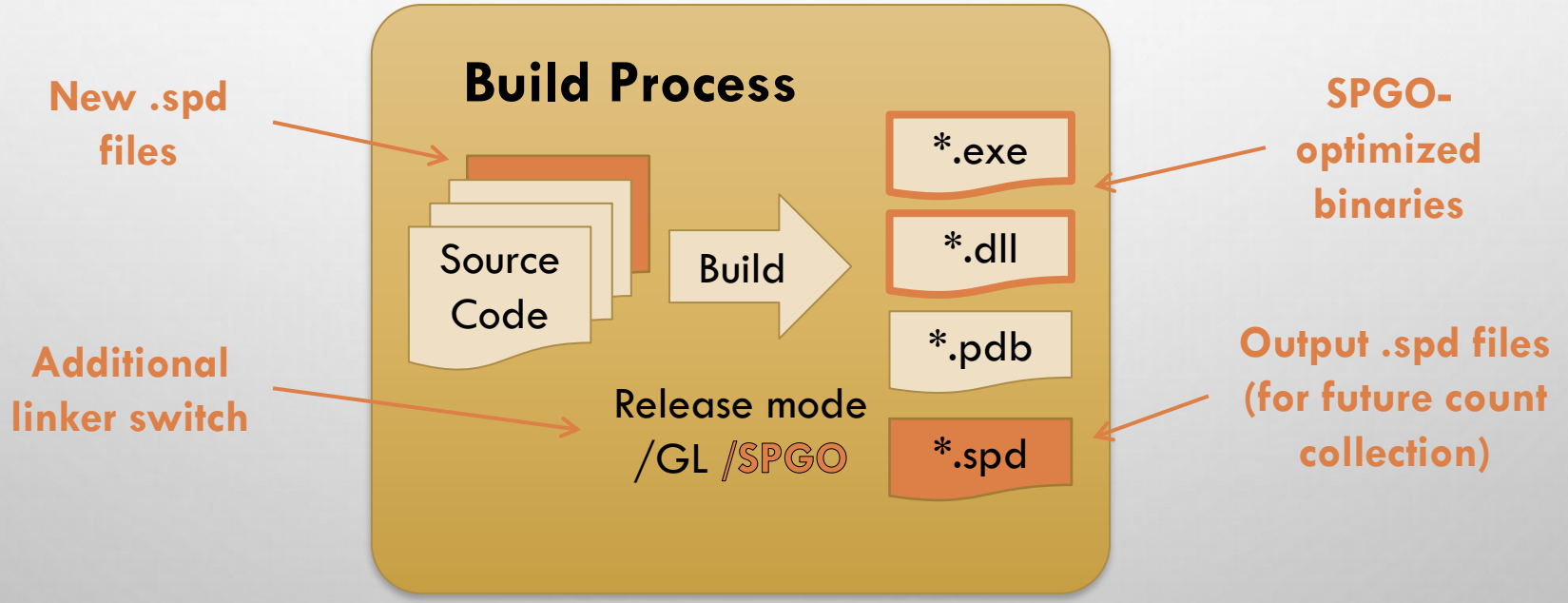


SPGO

https://aka.ms/spgo/tutorial & https://aka.ms/spgo/blog

Available on Visual Studio 2022 & later

5-15% performance improvements (YMMV)



https://aka.ms/spgo/tutorial & https://aka.ms/spgo/blog

GATHERING PROFILE COUNTS

1. Compile your Release binaries with /SPGO ← Produces 'empty' .spd
2. Collect an xperf trace
   - xperf.exe -on <arguments>
   - Run your application ← Produces .etl
   - xperf.exe -stop <arguments>
3. SPTAggregate.exe <arguments> ← Produces .spt
4. SPDConvert.exe <arguments> ← Integrate .spt into .spd, input to the next compilation

https://aka.ms/spgo/tutorial & https://aka.ms/spgo/blog

GATHERING PROFILE COUNTS

1. Compile your Release binaries with /SPGO
2. Collect an xperf trace
   - xperf.exe -on <arguments>
   - Run your application
   - xperf.exe -stop <arguments>
3. SPTAggregate.exe <arguments>
4. SPDConvert.exe <arguments>

Produces 'empty' .spd

Produces .etl

Produces .spt

Integrate .spt into .spd, input to the next compilation

These really matter for performance

https://aka.ms/spgo/tutorial & https://aka.ms/spgo/blog

PERFORMANCE RESULTS

Microsoft Azure

5% CPU usage reduction
15% IOPS improvement

7.9% CPU usage reduction

Code size reductions & perf wins

Microsoft Office

Perf wins in 'hard to move' scenarios

5-15% performance improvements

Significant code size reduction

Most effective on large codebases

Most effective with LBR profiling


Boosting Adobe Photoshop's Performance with MSVC and SPGO

Eric Brumer
Software Engineering Manager

Boosting Adobe Photoshop's Performance with MSVC and SPGO

Adobe and Microsoft worked together to improve Photoshop performance and responsiveness on Windows by combining MSVC's peak-performance build settings with Sample-based Profile-Guided Optimization (SPGO). The collaboration focused on real customer scenarios where latency matters most, including brush and stroke responsiveness, file open times, and other CPU-intensive operations that directly affect creative flow. Adobe's combined benchmarking showed **improvements of 20% on x64 and 13% on ARM64**, demonstrating how modern compiler technology can translate into a better experience for creatives using Photoshop on Windows.

~20% win on x64, ~7% win on ARM64

https://aka.ms/spgo/tutorial & https://aka.ms/spgo/blog

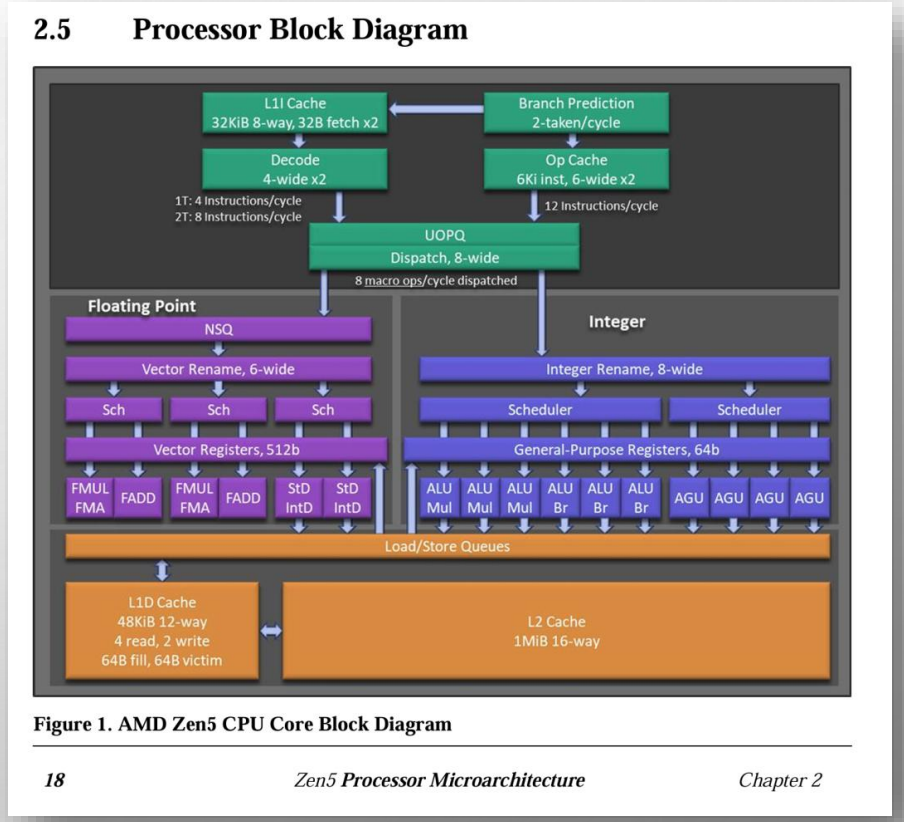
OPTIMIZATIONS

On hot paths

- Hyper-aggressive inlining
- Hyper-aggressive loop optimizations & vectorization
- Speculative devirtualization
- Code layout
- Data layout

On cold paths

- Inlining for code size
- Optimizations for code size
- Code separation



https://aka.ms/spgo/tutorial & https://aka.ms/spgo/blog

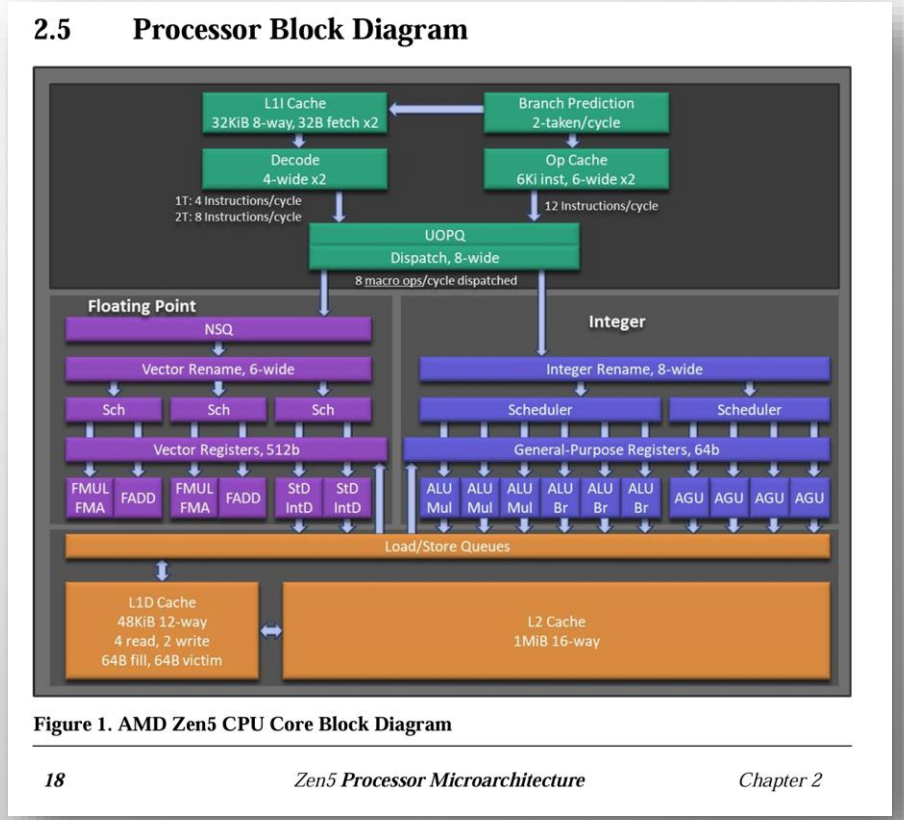
OPTIMIZATIONS

On hot paths

- Hyper-aggressive inlining
- Hyper-aggressive loop optimizations & vectorization
- Speculative devirtualization
- Code layout
- Data layout

On cold paths

- Inlining for code size
- Optimizations for code size
- Code separation



https://aka.ms/spgo/tutorial & https://aka.ms/spgo/blog

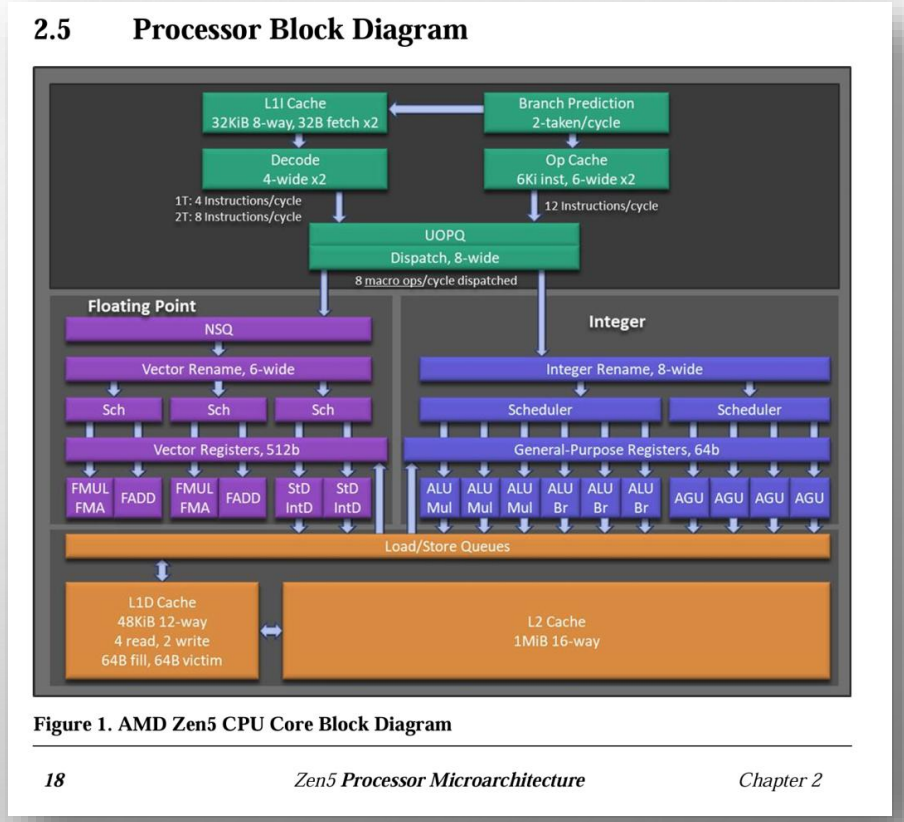
OPTIMIZATIONS

On hot paths

- Hyper-aggressive inlining
- Hyper-aggressive loop optimizations & vectorization
- Speculative devirtualization
- Code layout
- Data layout

On cold paths

- Inlining for code size
- Optimizations for code size
- Code separation



https://aka.ms/spgo/tutorial & https://aka.ms/spgo/blog

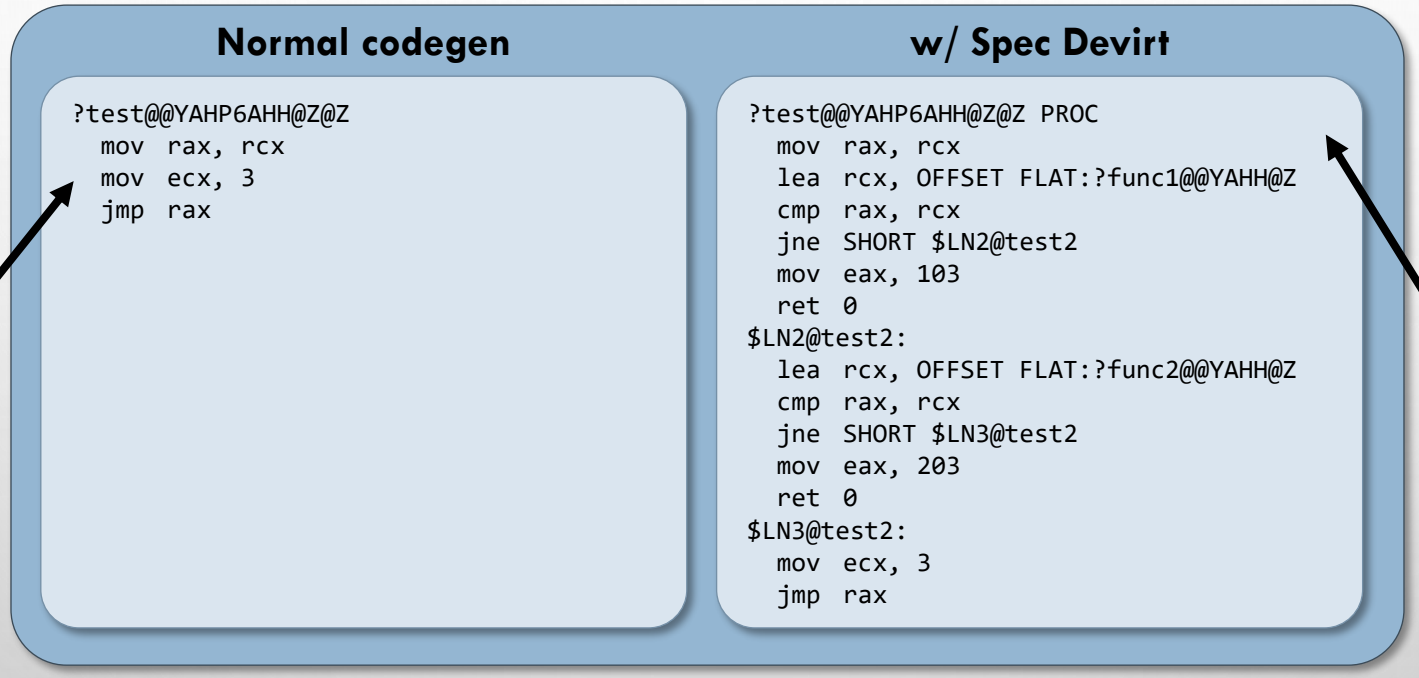
SPECULATIVE DEVIRTUALIZATION

```
typedef int (PFUNC)(int);

int func1(int x) {
    return x + 100;
}

int func2(int x) {
    return x + 200;
}

int test(PFUNC f) {
    return f(3);
}
```




```
typedef int (PFUNC)(int);

int func1(int x) {
    return x + 100;
}

int func2(int x) {
    return x + 200;
}

int test(PFUNC f) {
    if (f == func1)
        return func1(3);
    if (f == func2)
        return func2(3);
    return f(3);
}
```

Up to 30% perf win, despite code size increase

https://aka.ms/spgo/tutorial & https://aka.ms/spgo/blog

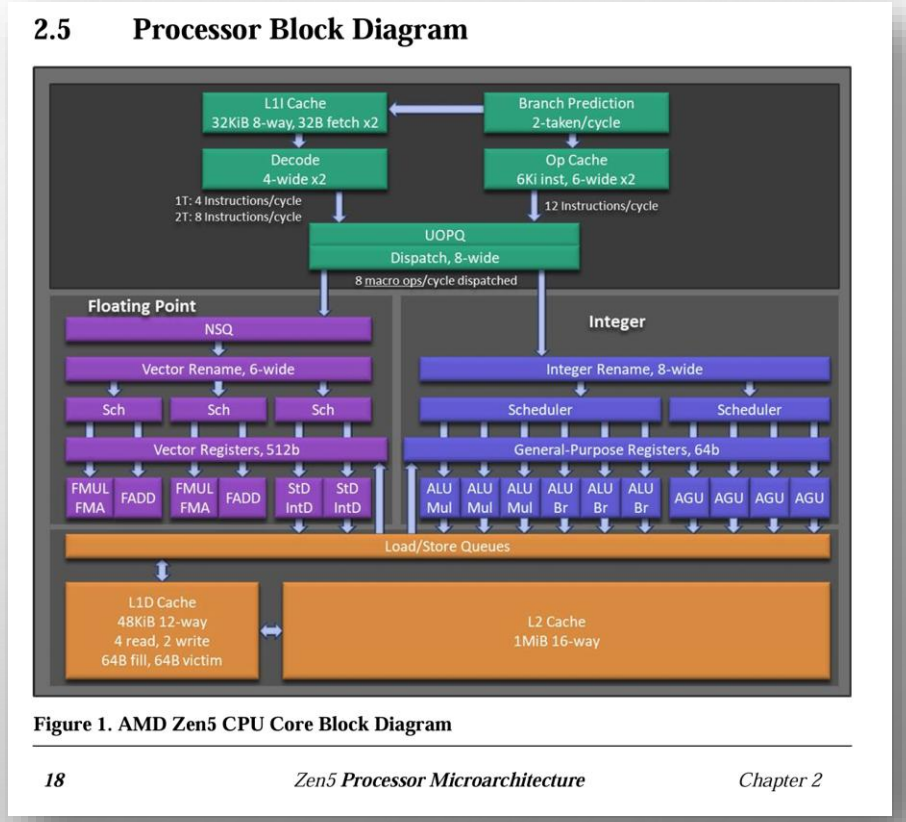
OPTIMIZATIONS

On hot paths

- Hyper-aggressive inlining
- Hyper-aggressive loop optimizations & vectorization
- Speculative devirtualization
- Code layout
- Data layout

On cold paths

- Inlining for code size
- Optimizations for code size
- Code separation



https://aka.ms/spgo/tutorial & https://aka.ms/spgo/blog

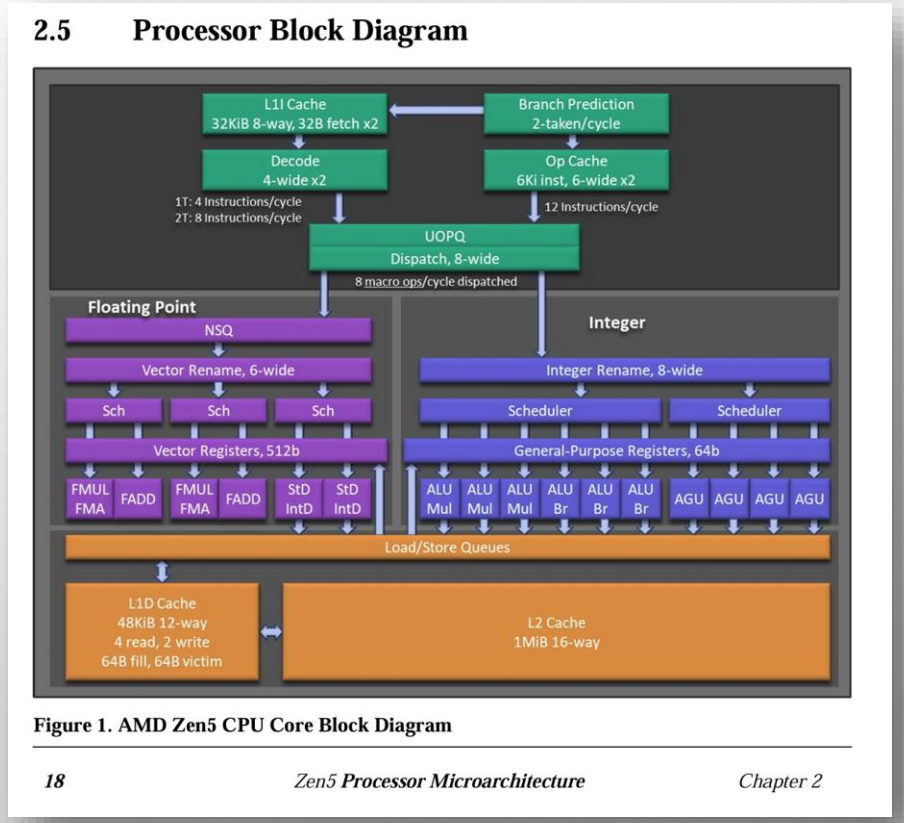
OPTIMIZATIONS

On hot paths

- Hyper-aggressive inlining
- Hyper-aggressive loop optimizations & vectorization
- Speculative devirtualization
- Code layout
- Data layout

On cold paths

- Inlining for code size
- Optimizations for code size
- Code separation



https://aka.ms/spgo/tutorial & https://aka.ms/spgo/blog

DEMO

AUTOFDO

- https://clang.llvm.org/docs/UsersManual.html#using-sampling-profilers
- https://github.com/google/autofdo

SPGO MISC

- PGO without instrumentation!
- Support for x64, x86, ARM64. Visual Studio 2022 & later: 14.44, 14.50, 14.51+
- /EMBED linker switch: SPD file embedded inside PDB
- SPT / SPD flexibility for complex profiling scenarios
- SPDDump.exe test.spd
- /d2:-SpgoCompileColdFunctionsWithoutProfileData
- Common issues on https://aka.ms/spgo/tutorial

https://aka.ms/spgo/tutorial & https://aka.ms/spgo/blog

MICROSOFT AT CPPCON 2026

Five sessions from the Microsoft C++ team. Come find us.

MON Sep 14	15:15 - 16:15	**The Journey to "/W4 /WX": How hard could it be?** Keith Stockdale
MON Sep 14	16:45 - 17:45	**C++ in the Age of AI: How Visual Studio Is Evolving** David Li, Augustin Popa
TUE Sep 15	09:00 - 10:00	**What's New for C++ in Visual Studio Code: C++ and AI Tooling for Your Build, Your Code, Your Workflow** Sinem Akinci, Ben McMorran · Tooling & Ecosystem
WED Sep 16	14:00 - 15:00	**Turbocharging Native Code Performance: Compiler Optimizations, Profile-guided Optimizations, and Beyond** Eric Brumer · Tooling & Ecosystem **You are here**
THU Sep 17	15:15 - 16:15	**The C++ Safety Issues Your Tools Can't See** Ion Todirel · Robotics & AI

cppcon2026.sched.com

Take the survey,

win a LEGO set

Scan the code with your phone. Takes about two minutes.



LEGO Creator 3-in-1
Retro camera



LEGO Creator 3-in-1
Game console



LEGO Botanicals
Orchid



LEGO Art
Milky Way galaxy

QUESTIONS?

EXTRA SLIDES

SLP OVERSIZED VECTORIZATION

```cpp
#include <cstdint>

// Compile with /O2 /arch:AVX for x64
void test(int16_t *s) {
    s[0] <<= 1;
    s[1] <<= 1;
    s[2] <<= 1;
    s[3] <<= 1;
}
```

Wins across CPU back-end

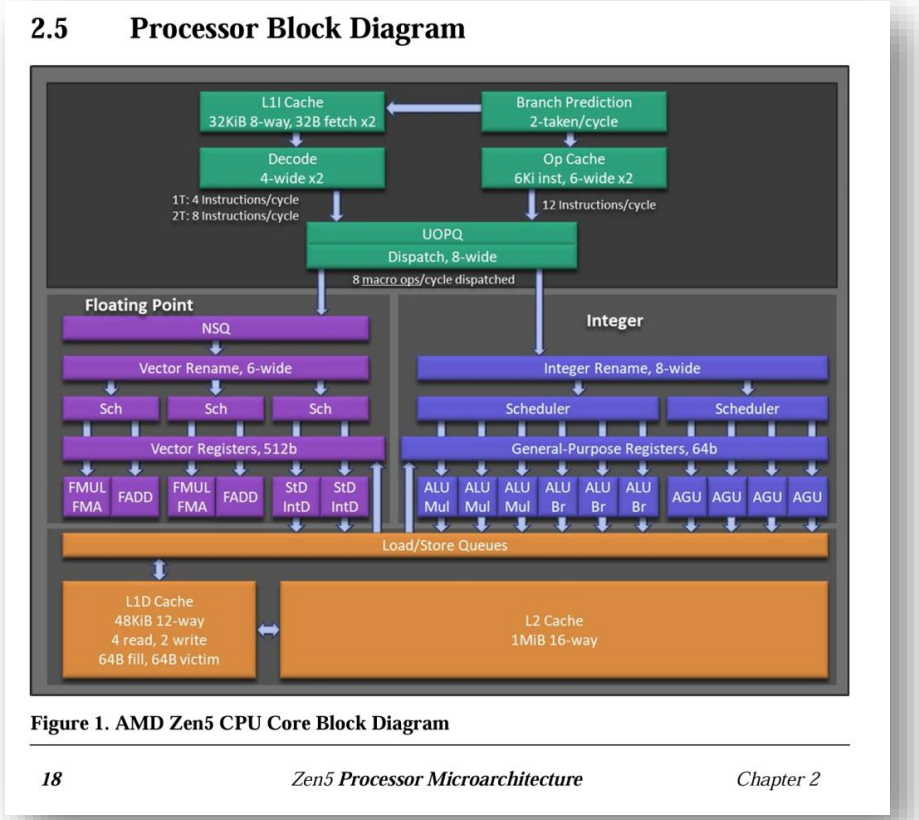
- SIMD generation

MSVC 14.50

```
void test(short *) PROC
  shl     WORD PTR [rcx], 1
  shl     WORD PTR [rcx+2], 1
  shl     WORD PTR [rcx+4], 1
  shl     WORD PTR [rcx+6], 1
  ret     0
```

MSVC 14.51

```
void test(short *) PROC
  vmovq   xmm0, QWORD PTR [rcx]
  vpsllw  xmm1, xmm0, 1
  vmovq   QWORD PTR [rcx], xmm1
  ret     0
```




```
vmovq   xmm0, QWORD PTR [rcx]    // xmm0 = { 0, 0, 0, 0, s[3],    s[2],    s[1],    s[0]    }
vpsllw  xmm1, xmm0, 1            // xmm1 = { 0, 0, 0, 0, s[3]<<1, s[2]<<1, s[1]<<1, s[0]<<1 }
vmovq   QWORD PTR [rcx], xmm1
```